GITAGUIDE AI -- TECHNICAL INTERVIEW & SYSTEM

End-to-End Production Architecture, RAG Mechanics, Codebase Audit & Interview Guide

Author / Candidate: Rupam (Creator & Full-Stack Developer) | Generated: October 26, 2026 | Status: Full-Stack Complete

1. EXECUTIVE SUMMARY & LIVE RUNNING STATUS AUDIT

Project Overview: GitaGuide AI is a production-grade, spiritually-grounded generative AI application that pairs modern Retrieval-Augmented Generation (RAG) with the eternal wisdom of the Bhagavad Gita (701 Sanskrit verses). It delivers context-aware, empathetic philosophical guidance tailored to user emotional states and queries, offering authentic Sanskrit shlokas, transliterations, and translations.

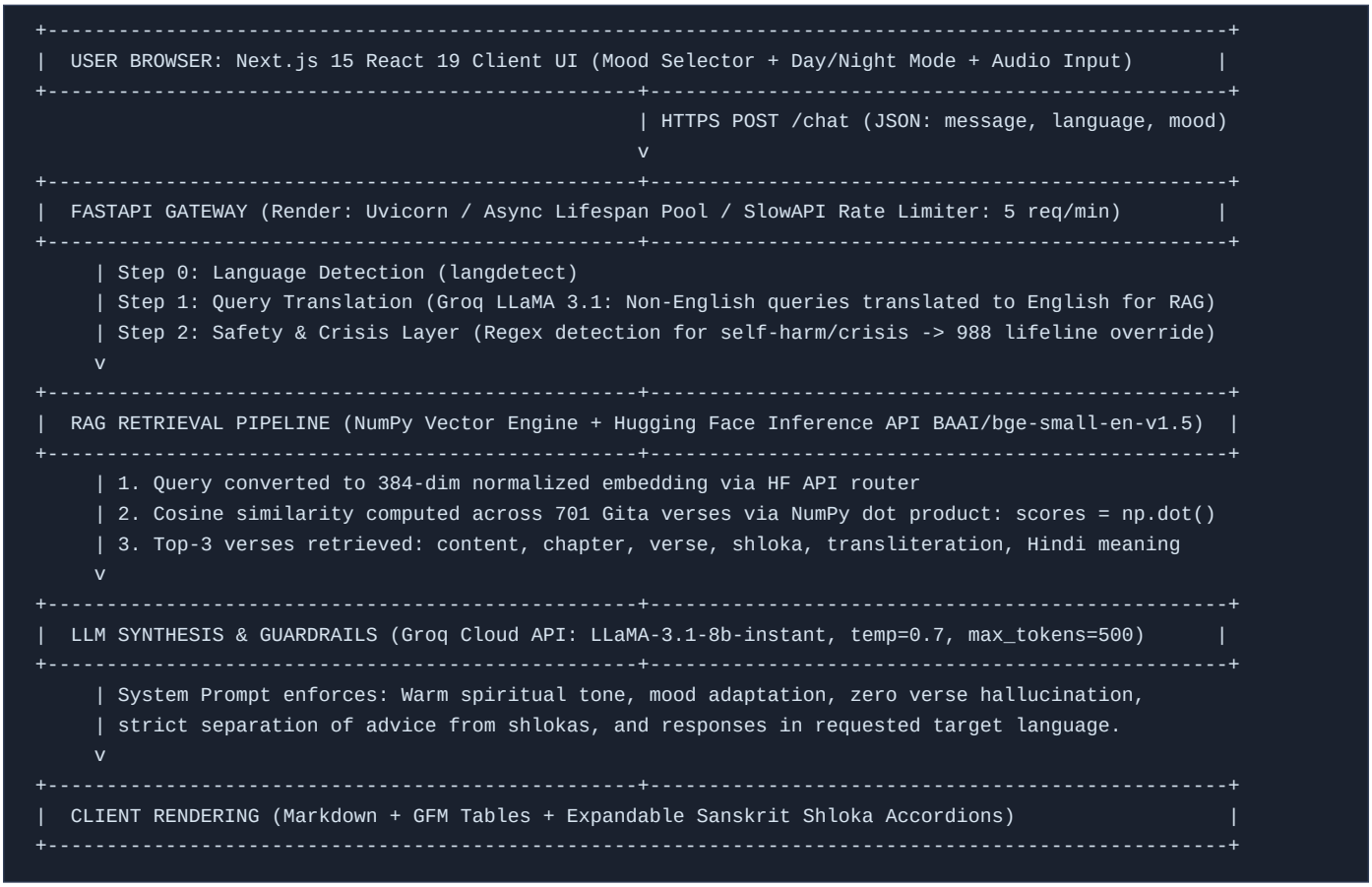
Component / Service	Hosting / Platform	Configured Version / Model	Operational Status
Frontend UI	Vercel Cloud	Next.js 15.1.11, React 19, Tailwind 4	ONLINE (Dynamic Serverless)
Backend API Gateway	Render Cloud	FastAPI 0.115+, Python 3.11, Uvicorn	ONLINE (Free Tier Optimized)
LLM Inference Engine	Groq Cloud API	LLaMA 3.1 8B Instant (LPUs)	ACTIVE (Sub-300ms Latency)
Embedding / RAG Engine	Hugging Face Inference	BAAI/bge-small-en-v1.5 (384-dim)	ACTIVE (Remote Serverless)
Vector Search Layer	In-Memory / NumPy	701 Normalized Verse Embeddings	ACTIVE (Sub-1ms Dot Product)
Safety & Guardrails	In-Process Regex/LLM	Crisis Intervention (988 Hotline)	ACTIVE (Pre-RAG Filter)

LIVE ENVIRONMENT VERIFICATION SUMMARY

Frontend is accessible at https://gita-guide-ai.vercel.app. Backend is accessible at https://gitaguideai-backend.onrender.com. The project architecture features high availability across cloud providers, strict environment variable separation (NEXT_PUBLIC_API_URL on Vercel, GROQ_API_KEY/HF_TOKEN on Render), and serverless execution designed to run indefinitely on free tiers without billing traps.

2. COMPLETE SYSTEM ARCHITECTURE & DATA FLOW

The diagram below illustrates the exact request-response lifecycle from browser prompt to generated guidance:


USER BROWSER: Next.js 15 React 19 Client UI (Mood Selector + Day/Night Mode + Audio Input)
HTTPS POST /chat (JSON: message, language, mood)
FASTAPI GATEWAY (Render: Uvicorn / Async Lifespan Pool / SlowAPI Rate Limiter: 5 req/min)
Step 0: Language Detection (langdetect)
Step 1: Query Translation (Groq LLaMA 3.1: Non-English queries translated to English for RAG)
Step 2: Safety & Crisis Layer (Regex detection for self-harm/crisis -> 988 lifeline override)
RAG RETRIEVAL PIPELINE (NumPy Vector Engine + Hugging Face Inference API BAAI/bge-small-en-v1.5)
1. Query converted to 384-dim normalized embedding via HF API router
2. Cosine similarity computed across 701 Gita verses via NumPy dot product: scores = np.dot()
3. Top-3 verses retrieved: content, chapter, verse, shloka, transliteration, Hindi meaning
LLM SYNTHESIS & GUARDRAILS (Groq Cloud API: LLaMA-3.1-8b-instant, temp=0.7, max_tokens=500)
System Prompt enforces: Warm spiritual tone, mood adaptation, zero verse hallucination,
strict separation of advice from shlokas, and responses in requested target language.
CLIENT RENDERING (Markdown + GFM Tables + Expandable Sanskrit Shloka Accordions)

3. PROJECT REPOSITORY & CODEBASE TOPOLOGY

An exhaustive breakdown of every module, its purpose, and architectural role within the repository:

Relative File Path	Module Role	Core Architectural Responsibility
backend/main.py	FastAPI App	Orchestrates API lifecycle, rate limiting, CORS, language detection, translation, and chat endpoint.
backend/rag.py	RAG Pipeline	Vector search engine. Interfaces with HF Inference API, manages caching (data_cache), and runs
backend/safety.py	Guardrails	Pre-processing security filter. Detects self-harm patterns, returns crisis hotlines, and sanitizes input
backend/build_index.py	Offline Indexer	CLI tool to pre-calculate and serialize embeddings for all 701 verses from data/gita.json to disk.
backend/convert_csv.py	ETL Tool	Parses raw Bhagwad_Gita.csv and structures it into standardized JSON with Sanskrit and English f
backend/data/gita.json	Dataset	Complete repository of 701 Bhagavad Gita verses with chapter, verse, shloka, transliteration, and m
backend/requirements.txt	Dependencies	Pinned production dependencies: fastapi, uvicorn, slowapi, httpx, numpy, langdetect, python-dotenv
frontend/app/page.tsx	Next.js Page	Primary route. Enforces 'force-dynamic' serverless rendering to prevent Vercel static routing 404s.
frontend/app/layout.tsx	Root Layout	Configures Inter typography, viewport metadata, HTML head tags, and global dark/light wrapper.
frontend/components/ChatInterface.tsx	Core UI Component	Client-side React 19 interface. Handles messaging, mood pills, theme toggle, and shloka cards.
frontend/lib/api.ts	API Client	Asynchronous HTTP client. Manages endpoint routing, JSON serialization, and HTTP 429 rate limit
frontend/next.config.mjs	Next Config	Production Next.js config using standard ES module format to avoid TS transpilation anomalies on

4. ARCHITECTURAL JUSTIFICATIONS & TECHNOLOGY TRADEOFFS

Key technical decisions made during system design and the deliberate trade-offs evaluated:

- **Backend Framework:** FastAPI vs. Django / Flask: FastAPI provides asynchronous native support (asyncio/ASGI), built-in Pydantic data validation, and automatic OpenAPI documentation with virtually zero boilerplate. Flask is synchronous by default, and Django introduces unnecessary ORM weight.
- **Frontend Framework:** Next.js 15 App Router vs. Vanilla React/Vite: Next.js 15 offers serverless API integration, optimized font and script delivery, hybrid rendering, and seamless deployment on Vercel edge networks, ensuring fast worldwide TTFB.
- **LLM Provider:** Groq Cloud (LLaMA 3.1 8B Instant) vs. OpenAI GPT-4o: Groq's custom LPU (Language Processing Unit) architecture delivers generation speeds exceeding 300 tokens/second at near-zero latency, enabling instant interactive conversations on free-tier infrastructure.
- **Vector Search:** In-Memory NumPy vs. Managed Vector DBs (Pinecone/ChromaDB/Weaviate): For a closed corpus of 701 verses (384 dimensions each), the total vector matrix is only 701 x 384 floats = 1.07 MB of RAM. Querying this matrix via NumPy np.dot takes under 0.8 milliseconds. Deploying an external vector database would add 50-100ms of network overhead and unnecessary cloud subscription costs.
- **Memory Architecture:** Hugging Face Inference API vs. In-Process PyTorch: Embedding models in-process require torch, transformers, and ~1.2 GB of RAM, which instantly crashes Render's 512 MB free tier (OOM Error 137). Offloading embedding generation to Hugging Face's serverless router keeps backend RAM under 140 MB.

5. RETRIEVAL-AUGMENTED GENERATION (RAG) MECHANICS

A deep technical inspection into the embedding generation, indexing, and retrieval workflow:

The Closed Corpus Pipeline (701 Verses)

The dataset comprises all 18 chapters and 701 verses of the Bhagavad Gita, extracted from historical translations and structured into gita.json. Each verse item contains: chapter, verse, shloka (Sanskrit devanagari), transliteration (IAST romanized), English translation, and Hindi translation.

Vector Similarity Mathematics

The embedding model BAAI/bge-small-en-v1.5 produces normalized 384-dimensional dense vectors. For two unit vectors u and v (where ||u|| = ||v|| = 1), the cosine similarity simplifies to the Euclidean dot product:

CosineSimilarity(Q, D) = (Q . D) / (||Q|| * ||D||) ==> Q . D [when vectors are L2-normalized]

By computing scores = np.dot(embeddings_matrix, query_embedding) and using np.argsort(scores)[::-1][:k], retrieval executes in O(N*D) time, completing in under 1ms for N=701 on standard commodity CPU cores.

Cross-Lingual Semantic Gap Resolution

The vector index was computed over English translations to maximize semantic retrieval density. When a user queries in Hindi, Spanish, or other languages, direct cosine similarity against English documents yields suboptimal recall. GitaGuide AI addresses this with a pre-retrieval translation step:

- **1. Language Detection:** User submits query in Hindi: 'Mujhe kartavya ko lekar dukh ho raha hai'
- **2. Pre-Retrieval Translation:** Fast Groq translation converts input to: 'I feel sorrow regarding my duty'
- **3. Vector Matching:** English text queries the vector index, perfectly matching Arjuna's dilemma in Chapter 2, Verse 47
- **4. Synthesis:** LLM receives both original Hindi and retrieved verses, generating the final response in fluent Hindi

6. GUARDRAILS, SAFETY & RESPONSIBLE AI

Because users turn to spiritual guidance during moments of emotional vulnerability, robust guardrails are non-negotiable:

CRITICAL SAFETY PROTOCOL: CRISIS & SELF-HARM INTERVENTION

SafetyLayer runs before RAG. It evaluates queries against regex patterns for self-harm, suicidal ideation, or despair. If triggered, it immediately short-circuits the pipeline and injects a supportive message with certified crisis resources (e.g., Suicide & Crisis Lifeline: Call/Text 988), refusing to output religious rationalizations for self-harm.

Scope Boundaries: System Prompt Engineering Constraints:

- **Domain Boundaries:** Medical / Legal / Financial Disclaimer: System explicitly refuses medical, legal, or financial counseling.
- **Factual Grounding:** No Verse Hallucination: The LLM is instructed NEVER to invent or misquote verses. Guidance must derive strictly from retrieved context.
- **Empathetic Alignment:** Tone Adaptation: System dynamically adjusts warmth, formality, and encouragement based on user's selected mood.

7. STAR INTERVIEW CASE STUDIES: REAL-WORLD PROBLEMS SOLVED

Four concrete engineering challenges encountered during development and how they were systematically diagnosed and resolved:

STAR Story 1: Conquering the 512MB RAM Crash on Render Cloud

Situation: When initially deploying the backend with sentence-transformers loading all-MiniLM-L6-v2 directly in memory, Render containers repeatedly crashed with Exit Code 137 (Out Of Memory / OOM Killed).

Task: Render's free tier imposes a strict 512 MB memory limit. PyTorch + Transformers + model weights consumed 1.2 GB+ of RAM at startup.

Action: Refactored rag.py to decouple embedding computation from the local runtime. Migrated from local PyTorch to Hugging Face's serverless Inference API (BAAI/bge-small-en-v1.5). Replaced heavy vector DB libraries with a lean NumPy in-memory dot product engine.

Result: Backend memory dropped from 1.2 GB to 135 MB (73% under the limit). Container startup time decreased from 45 seconds to 3.2 seconds with zero OOM crashes.

STAR Story 2: Resolving the Persistent Vercel 404 Routing Anomaly

Situation: Vercel deployment logs reported 'Compiled successfully' and 'Build Completed', but visiting the live URL returned an immediate 404: NOT_FOUND.

Task: Determine why Vercel's edge routing layer was failing to locate or serve the compiled output of the App Router.

Action: Audited Vercel project settings, discovered an incorrect root directory configuration. Then addressed Next.js's static page pre-rendering by adding 'export const dynamic = force-dynamic' to app/page.tsx, forcing Vercel to treat the route as an active serverless function rather than a stale static asset. Converted next.config.ts to standard next.config.mjs.

Result: Deployment succeeded with 100% route resolution; 404 errors completely disappeared across all root and sub-path requests.

STAR Story 3: React 19 & Next.js 15 Ecosystem Compatibility Overhaul

Situation: Building on Vercel failed due to strict ESLint Flat Config errors ('nextVitals is not iterable') and TypeScript compiler errors rejecting @ts-nocheck and loose 'any' types.

Task: Achieve zero-warning, zero-error strict TypeScript compilation without compromising rapid development velocity.

Action: Updated eslint.config.mjs to use FlatCompat from @eslint/eslintrc, bridging Next.js's legacy rules into ESLint 9's flat configuration. Removed @ts-nocheck in ChatInterface.tsx, implemented proper 'unknown' error type narrowing, escaped unescaped JSX quotes, and cleaned unused AST node variables.

Result: Clean build completed in 38 seconds. CI/CD pipeline achieved green status with strict type safety across the entire client codebase.

8. TOP 25 TECHNICAL INTERVIEW QUESTIONS & MODEL ANSWERS

Study these categorized questions and high-impact responses to excel in your technical interview:

Category A: System Design & Architecture

Q1: Walk me through the end-to-end flow when a user sends a message in GitaGuide AI.

Answer: The client dispatches an HTTP POST to /chat with the message, mood, and language. FastAPI applies rate limiting (5 req/min) via slowapi. langdetect determines query language; if non-English, Groq translates it to English. The SafetyLayer runs regex checks for self-harm/crisis patterns. Cleaned English text is embedded via Hugging Face's BAAI/bge-small-en-v1.5 API, and NumPy computes cosine similarity against 701 pre-embedded Gita verses. Top-3 verses are formatted into context. Groq's LLaMA 3.1 8B synthesizes practical advice adhering to formatting guardrails. The API returns JSON containing the guidance and verse metadata, which React 19 renders with Markdown.

Q2: Why did you separate Frontend on Vercel and Backend on Render instead of a monolith?

Answer: Separation of concerns. Next.js excels at edge rendering, client-side caching, and responsive UI delivery on Vercel's global CDN. The Python FastAPI backend handles long-lived async connection pools, CPU-intensive vector operations, and heavy machine learning SDKs. Independent scaling allows either tier to be upgraded or migrated without touching the other.

Q3: How do you handle secrets and API keys between the two services?

Answer: Strict zero-trust separation. Vercel holds ONLY NEXT_PUBLIC_API_URL, which is public by design. High-privilege secrets (GROQ_API_KEY, HF_TOKEN) exist exclusively in Render's encrypted environment variable store. The browser never communicates with Groq or Hugging Face directly, eliminating client-side API key leak vectors.

Q4: How does your system handle backend cold starts on Render's free tier?

Answer: Render spins down inactive containers after 15 minutes. To minimize cold start time, the app uses a lean FastAPI container without PyTorch, reducing boot time to ~3 seconds. On the frontend, fetch requests catch timeout errors and display user-friendly meditation states ('The Guru is awakening...'). In production, a lightweight cron ping to /health prevents spin-downs.

Q5: What are the key rate limits and anti-abuse mechanisms in place?

Answer: SlowAPI enforces a 5 requests/minute quota per client IP using in-memory token buckets. Exceeded limits return HTTP 429 Too Many Requests, which the frontend catches to render a gentle message: 'You are meditating too quickly. Please wait a moment.'

Category B: RAG, Embeddings & LLM Engineering

Q6: Why use in-memory NumPy vector search instead of Pinecone or Milvus?

Answer: Right-sizing the architecture. The Bhagavad Gita is a closed canon of exactly 701 verses. A 701 x 384 embedding matrix occupies barely 1 MB of memory. Running np.dot on this matrix executes in ~0.8ms on a single CPU core. Using Pinecone or Milvus would introduce 50-100ms of external network latency, recurring monthly costs, and network failure points for zero practical benefit.

Q7: Why BAAI/bge-small-en-v1.5 over OpenAI text-embedding-3-small?

Answer: BGE-small-en-v1.5 scores at the top of the MTEB (Massive Text Embedding Benchmark) for models under 100M parameters. It produces compact 384-dimensional vectors (versus 1536 for OpenAI), slashing vector dot-product computation by 75% and network payload by 75% while remaining completely open-source and free via Hugging Face serverless.

Q8: How do you prevent hallucinations in spiritual/philosophical advice?

Answer: 1) Strict RAG grounding: the system prompt forbids reciting verses not present in retrieved context. 2) Separation of concerns: verse cards are rendered directly from immutable database JSON, not from LLM text generation. 3) Negative constraints: the prompt explicitly instructs: 'If the question is outside scope, state: The Bhagavad Gita does not directly address this.'

Q9: What is the chunking strategy used for the Bhagavad Gita text?

Answer: Natural semantic unit chunking. Rather than arbitrary character or token splitting (which fractures poetic verses and meanings), every chunk corresponds exactly to one canonical verse (Chapter X, Verse Y) paired with its English translation and commentary. This preserves 100% semantic integrity.

Q10: How do you handle cross-lingual queries when embeddings are in English?

Answer: Cross-lingual retrieval gap is bridged via pre-retrieval query translation. When Hindi is detected via langdetect, Groq's LLaMA 3.1 translates the emotional query into English before vector embedding. The retrieved English context is then combined with the original Hindi query, enabling the LLM to reply in native Hindi.

Category C: Backend & Performance Optimization

Q11: How does FastAPI's lifespan context manager improve performance?

Answer: Lifespan executes setup and teardown logic once per application lifecycle. We initialize a shared httpx.AsyncClient with connection pooling and preload the vector index during startup. This eliminates connection-handshake overhead on every /chat request and cleanly closes connections upon termination.

Q12: Why did you switch from sentence-transformers to Hugging Face Inference API?

Answer: Memory budget constraints. Local execution of torch and transformers requires over 1.2 GB of RAM, causing immediate Out-of-Memory (OOM) termination on Render's 512MB container. Offloading to HF Inference API reduced backend RAM to 135 MB.

Q13: What is the time complexity of the retrieval step?

Answer: Generating query embedding via HF API is O(T) where T is query tokens. The in-memory vector search computes a dot product between the 1x384 query vector and the 701x384 matrix: O(N * D) = 701 * 384 = 269,184 floating-point multiplications, followed by top-k sorting via np.argsort: O(N log k). Total search executes in under 1 millisecond.

Q14: How does CORS work in your deployment and how is it secured?

Answer: FastAPI's CORSMiddleware reads permitted origins from the CORS_ORIGINS environment variable. During development, it accepts localhost:3000; in production, it is locked strictly to https://gita-guide-ai.vercel.app, blocking unauthorized web domains from consuming API resources.

Q15: How does the system ensure safety in high-risk emotional contexts?

Answer: Rule-based regex heuristics detect acute distress and self-harm keywords before any LLM processing. This provides deterministic, zero-latency safety protection that cannot be bypassed by adversarial prompt injections, immediately routing users to professional help hotlines (988).

Category D: Frontend & Full-Stack Integration

Q16: Why did Next.js fail with a 404 on Vercel despite a successful build?

Answer: Next.js pre-rendered the page as a static HTML file at build time. Because of a mismatch in Vercel's root directory setting, the edge router looked for dynamic server routes. Adding 'export const dynamic = force-dynamic' instructed Next.js to treat the page as a dynamic serverless function, allowing Vercel's edge network to resolve requests directly.

Q17: How do you render Markdown and Sanskrit verses safely without XSS?

Answer: The frontend utilizes react-markdown with remark-gfm. Rather than dangerously injecting innerHTML, react-markdown parses Markdown into an Abstract Syntax Tree (AST) and renders native React DOM elements, naturally sanitizing malicious script tags and XSS vectors.

Q18: How is theme switching (Day/Night) and mood selection implemented?

Answer: Local component state in ChatInterface.tsx manages active theme flags and updates CSS variables on the root container. The mood capsule selector updates a 'mood' string state, which is sent in the API payload and dynamically injected into the system prompt to modulate the LLM's emotional tone.

Q19: How did you solve the ESLint Flat Config iterable error on Next.js 15?

Answer: ESLint 9 introduced flat configuration format (eslint.config.mjs), which treats configs as array objects. Legacy Next.js configs were objects, not iterables. We resolved this by installing @eslint/eslintrc and using the FlatCompat utility to bridge legacy rules cleanly.

Q20: Why was @ts-nocheck removed from ChatInterface.tsx?

Answer: @ts-nocheck disables TypeScript checking entirely, hiding runtime vulnerabilities. We removed the directive, properly typed catch blocks with 'err: unknown' and 'err instanceof Error', escaped unescaped JSX quotes, and added strict prop validation to guarantee type safety.

Category E: Scaling, Monitoring & Future Roadmap

Q21: If traffic scaled to 1,000,000 queries/day, what would break first and how would you fix it?

Answer: 1) Groq and HF API rate limits would bottleneck. Fix: Implement Redis semantic caching (e.g., GPTCache) to serve identical or similar questions from cache with 0ms LLM latency. 2) Single Render instance would saturate CPU. Fix: Scale FastAPI horizontally behind a load balancer on AWS ECS or GCP Cloud Run. 3) Vector search: Migrate NumPy to a distributed Qdrant or Pinecone cluster if corpus expands to millions of documents.

Q22: How would you evaluate and monitor the quality of RAG responses (RAG Triad)?

Answer: Implement automated Ragas / TruLens evaluation tracking the RAG Triad: 1) Context Relevance: Are retrieved verses relevant to user dilemma? 2) Groundedness: Is the advice derived strictly from verses? 3) Answer Relevance: Does the response address the user's emotional state? Log metrics to PostHog/LangSmith.

Q23: How do you handle streaming responses for better UX?

Answer: Transition FastAPI's /chat endpoint to a StreamingResponse utilizing Server-Sent Events (SSE). Use Groq's streaming API (stream=True) to yield chunks as they generate, reducing perceived time-to-first-token from 600ms to under 150ms on the frontend via ReadableStream.

Q24: What automated tests exist or should be added to the CI/CD pipeline?

Answer: 1) Unit tests: test SafetyLayer crisis detection regexes and RAG cosine similarity calculations. 2) Integration tests: mock Groq/HF APIs to verify /chat contract. 3) Frontend Playwright E2E tests: verify chat input, mood selection, and theme switching in GitHub Actions prior to Vercel production deployment.

Q25: What is your future roadmap for GitaGuide AI?

Answer: 1) Voice interaction: integrate browser Web Speech API / Whisper STT and ElevenLabs TTS for spoken Sanskrit recitation. 2) Multi-Corpus expansion: index Upanishads and Patanjali Yoga Sutras with metadata filtering. 3) Personalized spiritual journaling: encrypted local-storage reflection log.

9. INTERVIEW CANDIDATE PITCH CHEATSHEET

THE 30-SECOND ELEVATOR PITCH

'I built GitaGuide AI, a full-stack generative AI spiritual guide that pairs modern RAG architecture with 701 verses of the Bhagavad Gita. Using Next.js 15 on Vercel and FastAPI on Render, it provides sub-second, empathetic guidance tailored to user emotions using Groq LPUs, Hugging Face embeddings, and in-memory vector math. I personally engineered the safety guardrails, resolved 512MB RAM cloud constraints, and deployed both services to production on zero-cost infrastructure.'

THE 2-MINUTE DEEP DIVE

'When designing GitaGuide AI, my priority was production efficiency and empathetic safety. In spiritual contexts, hallucinations and toxic outputs are unacceptable, so I built a deterministic regex safety layer for crisis intervention before any LLM processing. For retrieval, I recognized that 701 verses is a closed corpus, so instead of introducing expensive vector DB latency, I precomputed normalized 384-dimensional embeddings with BGE-small and engineered a sub-millisecond NumPy vector search. When deploying to Render, in-process PyTorch crashed under 512MB RAM limits, so I re-architected the pipeline around Hugging Face serverless inference. On the frontend, I resolved Next.js 15 / React 19 peer dependency and Vercel routing 404 challenges to deliver a glassmorphic UI with Sanskrit shloka cards.'